

Software Verification & Testing

Software Testing

- Testing should be performed with a mindset of finding bugs. This suspicious strategy(destructive approach) helps in finding more and more bugs.
- Software testing is a process that detects important bugs with the objective of having better quality software.
- Complete testing is not possible. Thus, the term ‘effective software testing’ is becoming more popular as compared to ‘exhaustive’ or ‘complete testing’.
- Exhaustive testing is not possible due to the following reasons:
 - It is not possible to test every possible input, as the input domain is too large.
 - There are too many possible paths through the program to test.
 - It is difficult to locate every design error.

Software Testing(Cont.)

- Effective software testing, instead of complete or exhaustive testing, is adopted such that critical test cases are covered first.
- There are different views on how to perform testing which have been categorized as schools of software testing, namely
 - analytical school,
 - standard school,
 - quality school,
 - context school, and
 - agile school.
- Software testing is a complete process like software development.

Software Testing Goals

Immediate Goals:

1. **Bug Discovery:** Find errors early in the development cycle for higher success rates.
2. **Bug Prevention:** Learn from discovered bugs to prevent future ones, minimizing errors in later stages.

Long-term Goals:

1. **Quality:** Achieved through rigorous testing, ensuring software meets high standards of correctness, integrity, and efficiency.
2. **Reliability:** Confidence in software functionality, directly affecting quality.
3. **Customer Satisfaction:** Satisfied users through comprehensive testing, meeting both specified and implied requirements.
4. **Risk Management:** Minimize risks by evaluating potential issues (e.g., cost, time, resources), and categorizing risk levels to guide testing activities.

Software Testing Goals(Cont.)

Post-Implementation Goals:

1. **Reduced Maintenance Costs:** Effective testing minimizes post-release errors, reducing costly maintenance.
2. **Improved Testing Process:** Analyze past projects to refine testing methods for future projects.

Software Testing Terminology

1. Failure

Occurs when the system or component does not perform as specified. It results from discrepancies between expected and actual behavior of the system.

2. Fault/Defect/Bug

A fault (or defect/bug) is the root cause of a failure, existing in any phase of the SDLC. Multiple faults may cause one failure, but not all faults lead to failures, as some might be hidden.

3. Error

A human mistake made during the development phase of SDLC. It causes bugs, which in turn lead to failures.

Software Testing Terminology(Cont.)

- The program fails to print the value of x because a condition prevents the while loop in Module A() from being executed. It is a **failure** of the program.
- The failure is identified as a **bug/defect/fault** caused by incorrect behavior in the program.
- The root cause of the issue is a **syntax error**, where a misplaced semicolon after the while loop prevents its execution.

```
Module A()  
{  
    ...  
    while(a > n+1);  
    {  
        ...  
        print("The value of x is", x);  
    }  
    ...  
}
```

Software Testing Terminology(Cont.)

4. Test Case

A documented procedure designed to test specific functionality, including:

- **Test Case ID:** Identification number.
- **Purpose:** Why the test is being conducted.
- **Preconditions:** Any required setup.
- **Inputs:** Specific values (e.g., 23 and 56).
- **Expected Outputs:** What should occur without failures. Test cases help find errors by comparing actual and expected results.

Test Case ID
Purpose
Preconditions
Inputs
Expected Outputs

Test case Template

5. Testware

The documentation produced during testing, such as test plans, case designs, and reports, which must be managed and updated.

Software Testing Terminology(Cont.)

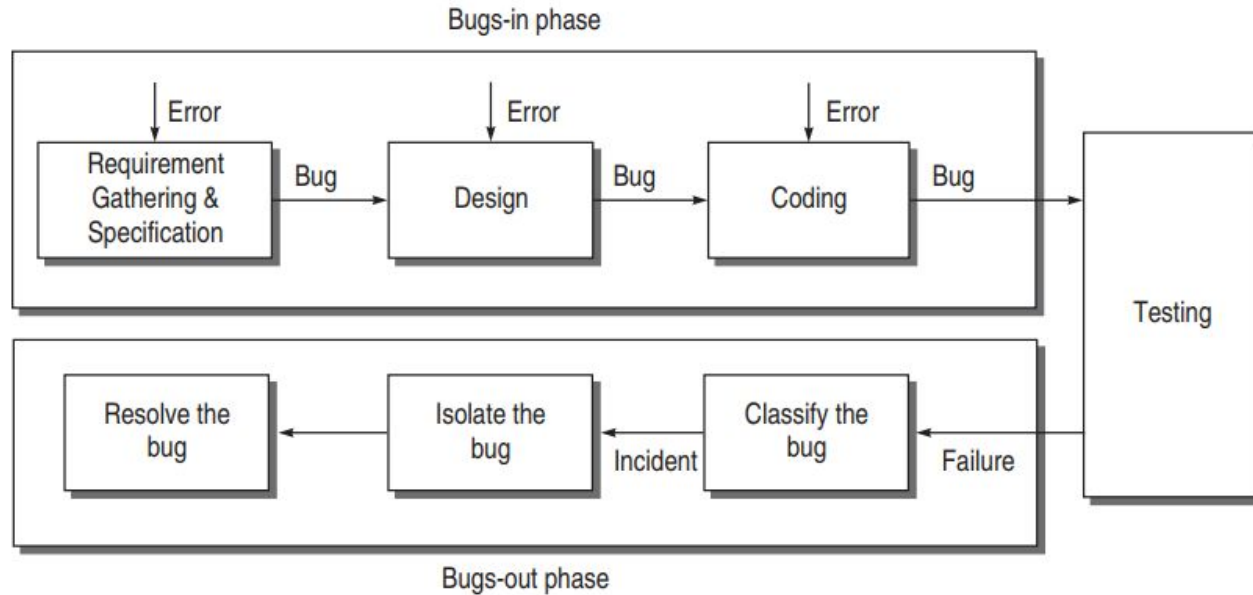
6. Incident

The symptom(s) of a failure that indicate a problem has occurred, often alerting the user to a failure.

7. Test Oracle

A mechanism used to judge the correctness of a test, typically by comparing actual results with expected outcomes.

Life Cycle of a Bug



Life Cycle of a Bug(Cont.)

1. Bugs-In Phase

- Errors are introduced during the SDLC and can go undetected to later phases.
- A bug is created when an error leads to failure but is not found in the current phase.
- Each phase may contain both errors and bugs passed on from previous stages, making bugs harder to detect if verification is not performed in earlier phases.

2. Bugs-Out Phase

- Occurs during testing when failures indicate the presence of bugs.
- **Bug Classification:** Bugs are categorized based on severity (e.g., critical, catastrophic, or trivial) to prioritize resolution.
- **Bug Isolation:** The process of locating the module or part of the system where the bug occurs, often using failure symptoms and back-tracing the design.
- **Bug Resolution:** Once isolated, the exact location of the bug is identified and corrected

States of a Bug in Its Life Cycle

1. New

The initial state when the bug is first reported by a tester.

2. Open

After the test leader approves the bug as genuine, it transitions to the "Open" state, awaiting further verification.

3. Assign

Once verified, the bug is assigned to the development team for validation and, if valid, a developer is tasked with fixing it.

4. Deferred

If the bug's priority is low, or it doesn't significantly affect the software, it is deferred for fixing in future releases.

States of a Bug in Its Life Cycle(Cont.)

5. Rejected

If the bug is determined to be invalid by the developer, it is rejected.

6. Test

Once fixed, the developer sends the bug back to the testing team for verification. The bug's state is now "Test."

7. Verified/Fix

The tester verifies the fix. If the bug is confirmed to be resolved, the status changes to "Verified."

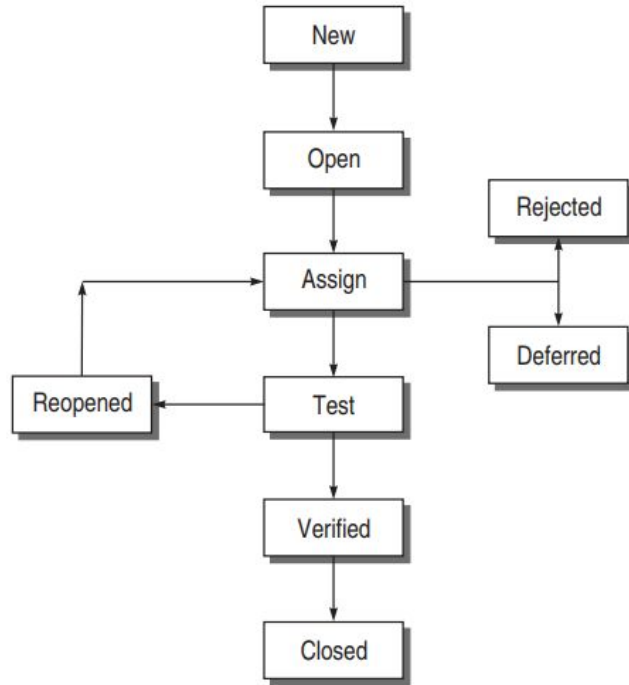
8. Reopened

If the bug persists after being fixed, or if it reappears after being closed, it is reopened and follows the life cycle again.

9. Closed

When the bug is confirmed to be fully fixed and no longer present, its state is changed to "Closed."

States of a Bug in Its Life Cycle(Cont.)



Why Do Bugs Occur?

1. To Err is Human

2. Bugs in Earlier Stages Go Undetected and Propagate

3. Reasons for Bugs

- **Miscommunication:** Poor communication in gathering requirements from the customer can introduce bugs.
- **Changing Requirements:** Constant changes can confuse development and testing teams, leading to bugs due to overlooked dependencies.
- **Module Interdependencies:** Ignoring the impact of changes in one module on others can result in bugs.
- **Resource Rescheduling:** New developers or changes in requirements can introduce bugs if proper coding standards and knowledge transfer are not followed.
- **Complexity:** Tracking and managing bugs, especially those with complex life cycles, can lead to more bugs.

Bug Classification Based on Criticality

1. Critical Bugs

- **Impact:** Stops or hangs normal software functioning.
- **Example:** System hangs after input in a sorting program.

2. Major Bugs

- **Impact:** Fails to meet functional requirements, but doesn't halt the software.
- **Example:** Sorting program displays incorrect output.

3. Medium Bugs

- **Impact:** Less critical, causing minor issues like redundant or truncated output.
- **Example:** Incorrect formatting in output.

4. Minor Bugs

- **Impact:** Mild, with no effect on functionality.
- **Example:** Typographical errors or misaligned printouts.

Bug Classification Based on SDLC Phases

1. Requirements and Specifications Bugs

- **Cause:** Errors during the requirement gathering phase.
- **Impact:** Misunderstood, incomplete, or inconsistent requirements leading to downstream issues.

2. Design Bugs

- **Types:**
 - **Control Flow Bugs:** Missing/unreachable paths in control flow.
 - **Logic Bugs:** Logical errors in design.
 - **Processing Bugs:** Mistakes in computation or data representation.
 - **Data Flow Bugs:** Errors like uninitialized or incorrectly used data.
 - **Error Handling Bugs:** Inadequate exception handling.
 - **Race Condition Bugs:** Unpredictable bugs due to timing.
 - **Boundary-Related Bugs:** Errors when values exceed or fall below expected limits.
 - **User Interface Bugs:** Issues related to usability or design.

Bug Classification Based on SDLC Phases(Cont.)

1. **Coding Bugs**

- **Cause:** Common coding mistakes like undeclared variables, typographical errors, or incorrect documentation.

2. **Interface and Integration Bugs**

- **External Interface Bugs:** Incorrect timing, sequence, or format issues with external systems.
- **Internal Interface Bugs:** Errors in internal module interactions, such as incorrect data transfers or parameter issues.

3. **System Bugs**

- **Cause:** Issues found when testing the system as a whole, such as performance or stress issues under load.

4. **Testing Bugs**

- **Cause:** Errors in the testing phase, like missed problems, ineffective test cases, or failure to verify fixes.

Testing Principles

- **Effective Testing over Exhaustive Testing**
 - Exhaustive testing is impractical; focus on effective testing to cover program logic and conditions adequately.
- **Testing Throughout SDLC**
 - Testing begins with specifications and continues until product release, not just after coding.
- **Independent Testing is Key**
 - Programmers often miss bugs in their own code. Independent testers or external testing teams are recommended for effective results.
- **Comprehensive Documentation**
 - Record all test cases, inputs, outputs, and bugs. Documented tests help in debugging and future projects.
- **Test with Invalid Inputs**
 - Negative testing with invalid inputs and unexpected behaviors often reveals errors.

Testing Principles(Cont.)

- **Tester Involvement in Reviews**
 - Testers should participate in specification and design reviews to ensure all behaviors are tested and no test cases are redundant.
- **Destructive Approach for Effective Testing**
 - Assume bugs exist and focus on uncovering them. A destructive mindset aids constructive testing.
- **Early Testing Saves Costs**
 - Start testing as soon as requirements are prepared to reduce the cost of late-stage bug fixes.
- **Error Clustering**
 - Focus on error-prone modules as bugs often appear in clusters.

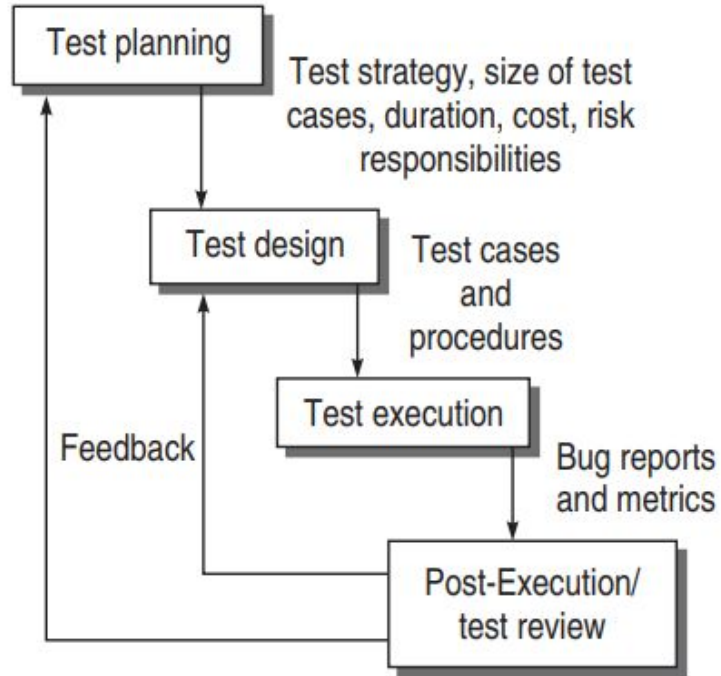
Testing Principles(Cont.)

- **Incremental Testing Strategy**
 - Begin testing at the unit/module level, then integrate and test the entire system.
- **Independent Team Testing**
 - Developers are biased towards their code; independent teams provide a more effective testing approach.

Software Testing Life Cycle (STLC)

- STLC is a systematic series of steps for effective software testing, akin to SDLC.
- Saves time and effort, uncovers more bugs, reduces project cost, and improves schedule by involving testers early.
- Helps in measuring milestones.

Software Testing Life Cycle (STLC)



STLC Phases - Test Planning

Goal: Create a roadmap addressing testing strategy, resources, schedules, responsibilities, risks, and priorities, aligned with project planning.

Key Activities:

- Define the test strategy.
- Estimate test cases, duration, and cost.
- Plan resources (manpower, tools, documents).
- Identify risks.
- Set test completion criteria.
- Select methodologies, techniques, and tools.
- Define reporting procedures, bug classification, severity levels, and metrics.

STLC Phases - Test Planning(Cont.)

Outputs:

- Develop test plan document for each testing level.
- Create test case format and plans for all SDLC phases.
- Identify test cases for automation.
- Prioritize test cases by importance.
- Define stress and performance testing areas.
- Plan regression testing cycles.

STLC Phases - Test Design

- Test design is a well-planned process that involves creating effective test cases to ensure comprehensive testing.
- The test design phase results in a comprehensive *Test Design Specification* document detailing input/output specs, environment needs, and procedural requirements.

STLC Phases - Test Design(Cont.)

Key Activities in Test Design:

1. **Determine Test Objectives:**
 - Identify key elements to test using reference materials (e.g., software requirements, design docs).
 - Prioritize objectives based on scope and risk.
2. **Prepare Test Items List:**
 - Convert objectives into a list of specific items to be tested.
3. **Map Items to Test Cases:**
 - Use a matrix to link test cases with test items.
 - Helps identify major scenarios, reduce redundancy, and find gaps.
4. **Design Test Cases:**
 - Analyze the program (functional/structural levels).
 - Follow Tsuneo Yamaura's rule: *Cover all features but avoid too many test cases.*

STLC Phases - Test Design(Cont.)

Attributes of a Good Test Case:

- Focus on critical/high-risk requirements.
- High probability of finding errors.
- Non-overlapping, unique functionality.
- Modular design for reusability and simplicity.
- Detects undiscovered errors effectively.

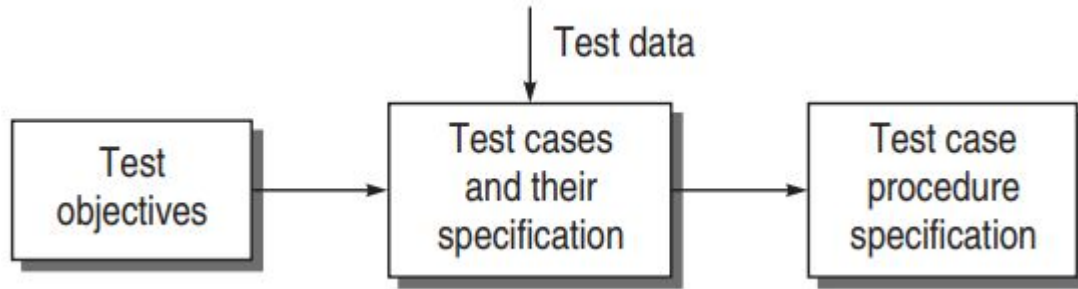
STLC Phases - Test Design(Cont.)

Test Case Design Techniques:

1. **Black-Box Testing:**
 - Based on external behavior; internal system details unknown.
2. **White-Box Testing:**
 - Based on internal system logic.

Steps in Test Design:

1. **Create Test Cases & Data:**
 - Include objectives, inputs, expected outputs, and careful test data selection.
2. **Set Up Test Environment:**
 - Specify hardware, software, tools, and configurations.
3. **Develop Test Procedure Specification:**
 - Document sequenced steps for executing test cases.



Test case design steps

STLC Phases - Test Execution

- Executes all test cases for **verification** (end of each SDLC phase) and **validation** (after module completion).
- Decision to use **manual or automated execution** lies with the test team.
- Results documented in **test incident reports, logs, status, and summary reports**.

STLC Phases - Test Execution(Cont.)

Post-Execution/Test Review

- Bugs are reported to developers for resolution and feedback.
- Developer activities:
 1. **Understanding the bug:** Analyze and locate the issue.
 2. **Reproducing the bug:** Confirm by replicating the failure.
 3. **Analyzing the cause:** Debug symptoms to find the error source.
 4. Fix the bug and notify the testing team.

STLC Phases - Test Execution(Cont.)

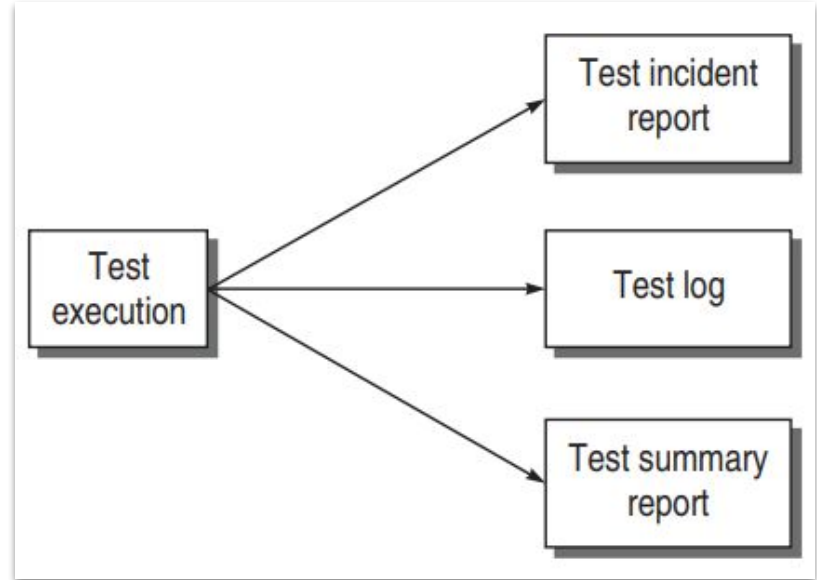
Final Testing & Review

- Modified software is re-tested (manual and automated).
- Analyze results through:
 - **Reliability analysis:** Ensure predefined goals are met; decide release.
 - **Coverage analysis:** Evaluate testing completion criteria.
 - **Defect analysis:** Identify risks and improve quality.

STLC Phases - Test Execution(Cont.)

Test Execution Level	Person Responsible
Unit	Developer of the module
Integration	Testers and Developers
System	Testers, Developers, End-users
Acceptance	Testers, End-users

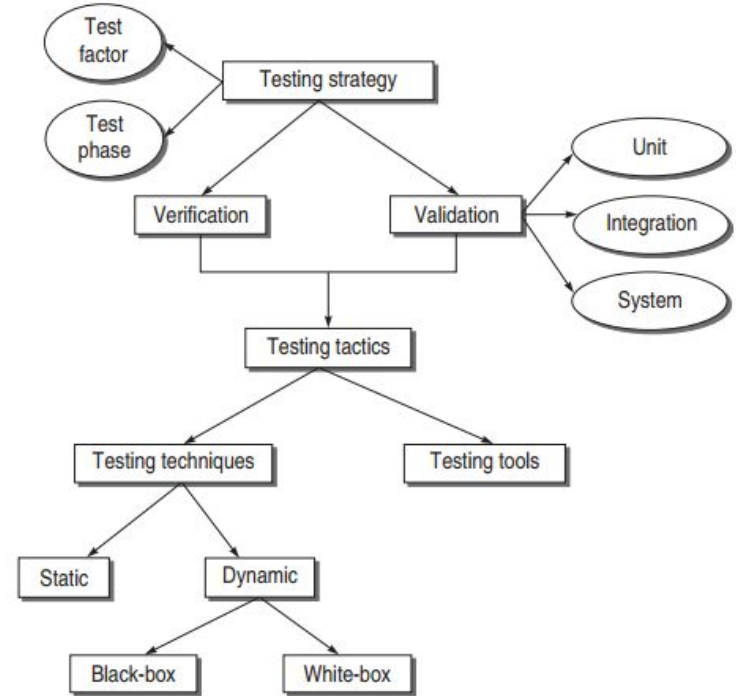
Testing level vs responsibility



Documents in test execution

Software Testing Methodology

Software testing methodology is the organization of software testing by means of which the test strategy and test tactics are achieved.



Software Testing Strategy

A software testing strategy outlines a structured plan for conducting the testing process in a series of well-defined steps. It serves as a roadmap detailing specific activities to be performed by the test team to achieve the testing goals, primarily focused on **risk reduction**.

Key Components of Testing Strategy:

1. Test Factors:

- These are risk factors or issues associated with the system under development.
- They must be identified, prioritized, and mitigated to an acceptable level during testing.

2. Test Phases:

- These define the stages of the SDLC where testing occurs.
- Testing strategies vary based on the SDLC model (e.g., different approaches for waterfall vs. spiral models).

Software Testing Strategy - Test Strategy Matrix

- A tool for identifying key concerns in test planning and execution, serving as input to develop a testing strategy.
- **Components:**
 - **Rows: Test Factors** - Ranked factors specific to the system, from most to least significant.
 - **Columns: Test Phases** - System development phases based on the adopted model.
- **Steps to Prepare:**
 - **Select and Rank Test Factors:** Identify and prioritize relevant test factors.
 - **Identify Test Phases:** List development phases as matrix columns.
 - **Address Risks:**
 - Identify risks (e.g., delays in budget, equipment, software availability).
 - Assess impact and mitigate with strategies.
 - Define concerns as **questions** to guide high-level testing focus.
- **Purpose:** Ensure test concerns are addressed in the appropriate phase while minimizing risks.

Software Testing Strategy - Test Strategy Matrix(Cont.)

Test Factors	Test Phase					
	Requirements	Design	Code	Unit test	Integration test	System test

Creating Test Strategy

Test Factors	Test Phase					
	Requirements	Design	Code	Unit test	Integration test	System test
Portability	Is portability feature mentioned in specifications according to different hardware?					Is system testing performed on MIPS and INTEL platforms?
Service Level	Is time frame for booting mentioned?	Is time frame incorporated in design of the module?				

Test Strategy Matrix

Test Strategy - Designing a New Operating System

1. Select and Rank Test Factors

- Key factor: **Portability** – Effort to ensure compatibility across hardware (e.g., MIPS, INTEL).

2. Identify Test Phases

- Determine test phases impacted by test factors (e.g., Requirements, Design, Unit Test, System Test).

3. Identify Risks for Each Test Phase

- Example Risk: *“Is testing successfully performed on INTEL and MIPS platforms?”*

4. Plan Test Strategy for Identified Risks

- Develop strategies to mitigate risks and focus on:
 - Cost overruns
 - Schedule delays
 - Critical errors and failures

Development Of Test Strategy

Testing Starts Early:

Testing begins with the first step of the Software Development Life Cycle (SDLC) and continues until project implementation.

Test Strategy:

- Begins at the **component level** and progresses to **system integration**.
- Involves both **Verification** and **Validation**—essential for all testing processes.

Verification:

- Ensures the software aligns with specifications at each development phase.
- Detects and prevents defects early in the SDLC.
- Scope decreases as development progresses.
- Focus: *"Are we building the product right?"*

Development Of Test Strategy(Cont.)

Validation:

- Tests the complete system for conformance with customer expectations.
- Takes precedence in later stages of SDLC.
- Focus: *"Are we building the right product?"*

Combination of Verification & Validation:

A balanced testing process integrates both to ensure quality and customer satisfaction.

Development Of Test Strategy(Cont.)

Testing Scope

- **Verification:** Plan, design, and code phases; decreases as the system progresses.
- **Validation:** Begins post-verification, focusing on modules, integration, and the final system.

Key Testing Stages

- **Module Testing:** Validate individual modules after development.
- **Integration Testing:** Ensure correct interaction between integrated modules.
- **System Testing:** Test the entire system to identify bugs and ensure conformance with requirements.

Testing Life Cycle Model

Verification and Validation (V&V): Core components of the testing process; form the foundation of test strategy.

Testing Life Cycle Model:

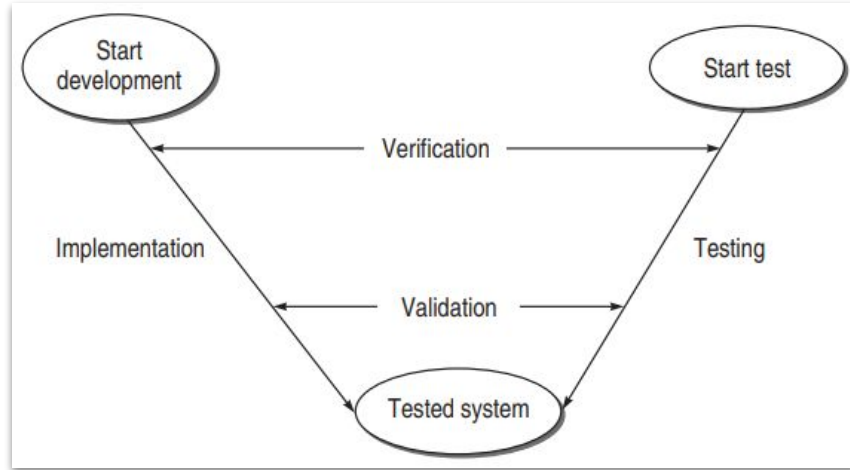
- Involves continuous testing during system development.
- Inspections at predetermined points identify defects early.
- Depends on deliverables at specified stages of the development life cycle.
- Maps various types of testing activities.

Testing Life Cycle Model(Cont.)

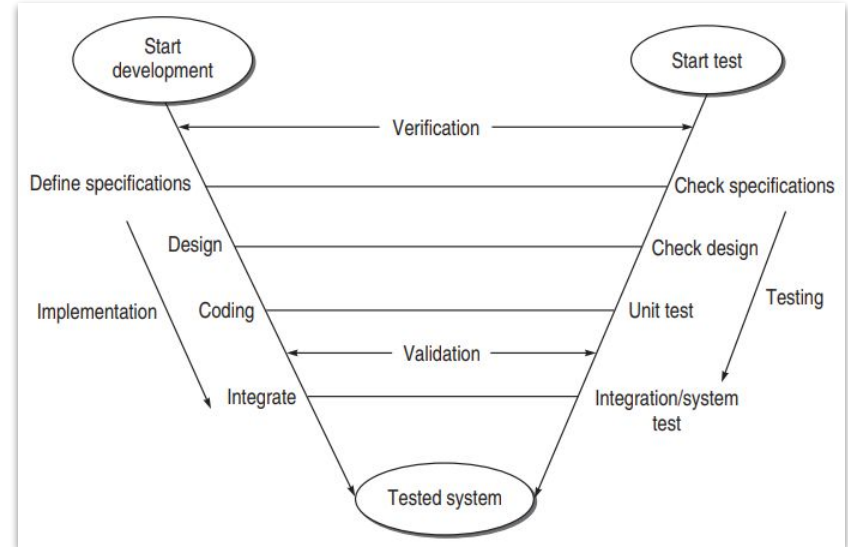
V-Testing Life Cycle Model:

- Testing and development proceed concurrently from the project start.
- **Left arm of the V:** Development phases.
- **Right arm of the V:** Corresponding testing stages.
- Early SDLC stages emphasize **verification**; later stages focus on **validation**.
- Key goals:
 1. **Verification** of each SDLC step.
 2. **Validation** of the verified system at the end.

V-Testing Life Cycle Model



V-testing model



Expanded V-testing model

Validation Activities

Validation involves three levels of testing:

1. **Unit Testing**

- Validates the smallest module of the system.
- Ensures the module behaves as per functional specifications.
- Critical to detect and fix bugs early, preventing latent bugs from reaching the customer.

2. **Integration Testing**

- Combines unit-tested modules to test their interaction.
- Focuses on interface specifications between modules.
- Ensures proper data structures, messages, and formats are used for seamless integration.
- Requires a systematic approach guided by the system design and module availability.

Validation Activities(Cont.)

3. System Testing

- Tests the entire integrated system.
- Covers various user scenarios and environments.
- Validates the system against requirement specifications to ensure overall functionality.

Testing Tactics

- Complete or exhaustive testing is impossible; focus on **effective testing** to maximize bug detection and testing domain coverage.
- Prioritize test cases with the highest probability of detecting the most errors within given constraints (time, cost, resources).

Software Testing Techniques:

- Methods for designing and implementing effective test cases.
- Categorized into:
 - **Static Testing:**
 - Examines structural characteristics (e.g., source code, design) without executing code.
 - Example: Code review.

Testing Tactics(Cont.)

Software Testing Techniques - Dynamic Testing:

- Executes code to test its functionality with user-provided inputs.
- Divided into:
 - **Black-box Testing:**
 - Focuses on input-output functionality.
 - Ignores internal processing details.
 - Used in system testing (validation).
 - **White-box Testing:**
 - Focuses on internal logic, paths, and code execution.
 - Complements black-box testing.
 - Used in unit testing (verification).

Testing Tactics(Cont.)

Testing Tools:

- Automate testing techniques to enhance efficiency.
- Combine tools and techniques for an effective test process.
- Key: Understand techniques before selecting appropriate tools.

Considerations in Developing Testing Methodologies

1. **Determine Project Risks**
 - Identify high-priority risks and their consequences.
 - Collaborate with teams familiar with business risks.
2. **Determine the Type of Development Project**
 - Consider risks based on the project type: new development, maintenance, or purchased software.
3. **Identify Test Activities According to SDLC Phase**
 - Recognize testing activities for all phases of the SDLC.
 - Begin testing after identifying risk factors in each phase.

Considerations in Developing Testing Methodologies(Cont.)

- **Build the Test Plan**

- Include the following in the test plan:
 - **Environment** and pre-test setup
 - **Objectives** and scope
 - **Test team**, schedule, and budget
 - **Resources**: equipment, software, personnel
 - **Materials**: system documentation, test inputs, tools
 - **Business and structural requirements** to be tested
 - Specific **testing techniques** for each SDLC phase

Debugging

Debugging

- Software testing lacks maturity as a discipline, making debugging an unplanned activity in most projects.
- Debugging is often viewed as an art rather than a structured technique, yielding better results.
- Analyzing bug history is crucial, as similar projects often encounter similar bugs.
- Critical thinking in testing and debugging supports the artistic approach.
- Debugging starts with analyzing failure symptoms to trace the actual error.

Debugging Process

Goal of Debugging

- Identify the failure using symptoms.
- Locate and correct bugs or errors.

Debugging Process(Cont.)

Steps in Debugging:

1. **Check Output:**
 - Compare actual vs. expected output from test cases.
 - Match = Success, Mismatch = Failure to analyze.
2. **Analyze Failure:**
 - Investigate the cause of failure.
 - If symptoms are insufficient, conduct additional testing for more clues.
3. **Identify and Trace Bug:**
 - Use clues to find the bug's cause and locate the error.
4. **Correct Errors:**
 - Fix the bug at its identified location.
5. **Perform Regression Testing:**
 - Validate corrections after every modification to ensure stability.

Debugging Techniques - Debugging with Memory Dump

- This technique involves analyzing a printout of registers and memory locations (in octal or hexadecimal format) to identify program bugs.
- Observes relevant program data via memory locations and registers.
- Inefficient and used as a last resort.
- **Drawbacks:**
 1. Difficulty in mapping storage locations to source program variables.
 2. Overwhelming and mostly irrelevant data.
 3. Limited to a single instant, showing only the program's static state.

Debugging Techniques - Debugging with Watch Points

Purpose of Watch Points

Identify bugs by examining program behavior and variable states at specific execution points.

Methods for Debugging

1. Output Statements
2. Breakpoint Execution
3. Single Stepping
4. Step-Into and Step-Over

Debugging Techniques - Debugging with Watch Points(Cont.)

- **Output Statements**

- Insert output statements at various watch points to check conditions or variables.
- Drawbacks:
 - Requires code changes, possibly masking or introducing errors.
 - Forgetting to remove statements can cause further issues.

- **Breakpoint Execution**

- **Breakpoints:** Automated watch points that stop execution for examination without altering the code.
- **Advantages over Output Statements:**
 - No recompilation needed after inserting breakpoints.
 - Easy to remove breakpoints.
 - Allows temporary suspension of program execution for variable or condition review.

Debugging Techniques - Debugging with Watch Points(Cont.)

- **Types of Breakpoints:**
 - **Unconditional:** Stops execution unconditionally.
 - **Conditional:** Stops execution based on a Boolean condition.
 - **Temporary:** Used once, automatically removed after stopping execution.
 - **Internal:** Invisible, used internally by the debugger.

Debugging Techniques - Debugging with Watch Points(Cont.)

- **Single Stepping**
 - Step through the program one instruction at a time to observe variable status after each step.
- **Step-Into and Step-Over**
 - **Step-Into:** Enter a function and stop at its first executable line.
 - **Step-Over:** Treat a function call as a single operation and proceed to the next source line.

Backtracking for Debugging

Backtracking is a logical approach to debugging, involving systematic tracing and isolating errors to identify the root cause.

Requirements:

- Knowledge of system design and logical flow.
- Ability to interpret DFDs.
- Manual observation of the source code.

Effectiveness: Pinpoints error locations quickly with a strong understanding of program logic.

Backtracking for Debugging

Steps for Backtracking:

1. **Observe Symptoms:**

- Identify failure at the output.
- Locate the site where symptoms appear (e.g., a function not working or a module not sending data).

2. **Trace Backwards:**

- Examine the source code starting from the symptom site.
- Move to higher levels of design abstraction to uncover the bug.

3. **Isolate the Module:**

- Use data flow diagrams (DFDs) to identify the affected module.

4. **Pinpoint the Bug:**

- Perform logical backtracking within the isolated module to locate the actual error.

Unit Testing

JUnit

JUnit 5 is a modern testing framework for Java, emphasizing modularity and extensibility. Key components include:

- **JUnit Platform:** Launches test frameworks.
- **JUnit Jupiter:** Provides new annotations and programming models.
- **JUnit Vintage:** Supports older JUnit versions.

Lifecycle Annotations

1. By default, JUnit creates a new instance of the test class for each test method (i.e., `TestInstance.Lifecycle.PER_METHOD`).
2. `@TestInstance(TestInstance.Lifecycle.PER_CLASS)`, JUnit creates a single instance of the test class for all test methods in that class.
3. `@BeforeAll`: Runs once before all tests.
4. `@AfterAll`: Runs once after all tests.
5. `@BeforeEach`: Runs before each test.
6. `@AfterEach`: Runs after each test.

Assertions

- **assertNull(Object)**: Ensures the object is null.
- **assertNotNull(object)**: Verifies that the object is not null.
- **assertEquals(expected, actual, message)**: Verifies that the expected result matches the actual value.
- **assertNotEquals(expected, actual, message)**: Verifies that two values are not equal.
- **assertThrows(expectedException, executable)**: Ensures that a specific exception is thrown during the test.
- **assertDoesNotThrow(executable)**: Asserts that the code doesn't throw any exception.
- **fail(message)**: Forces a failure with a custom message.

Assertions(Cont.)

- **assertTrue(condition)**: Asserts that the condition is true.
- **assertFalse(condition)**: Asserts that the condition is false.
- **assertSame(expected, actual)**: Verifies that the two references point to the same object.
- **assertNotSame(expected, actual)**: Verifies that the two references doesn't point to the same object.
- **assertArrayEquals(expectedArray, actualArray)**: Compares two arrays.
- **assertLinesMatch(expectedLines, actualLines)**: Compares two lists of lines, ensuring they match.
- **assertTimeout(timeout, executable)**: Verifies that the code executes within the specified time duration.

```

public class ACalculator {
    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }

    public int multiply(int a, int b) {
        return a * b;
    }

    public int divide(int a, int b) {
        if (b == 0) throw new ArithmeticException("Division by zero");
        return a / b;
    }

    public String getNull() {
        return null;
    }
}

```

```

import ...

@TestInstance(TestInstance.Lifecycle.PER_CLASS)
public class ACalculatorTest {
    private ACalculator calculator;

    @BeforeAll
    void setUpAll() {
        System.out.println("Initializing resources before all tests.");
    }

    @AfterAll
    void tearDownAll() {
        System.out.println("Cleaning up resources after all tests.");
    }

    @BeforeEach
    void setUp() {
        System.out.println("Setting up before each test.");
        calculator = new ACalculator();
    }

    @AfterEach
    void tearDown() {
        System.out.println("Cleaning up after each test.");
    }

    @Test
    void testAssertions() {...}
}

```

@Test

void testAssertions() {

assertNull(calculator.getNull(), message: "Expected a null value");

assertNotNull(calculator, message: "Calculator instance should not be null");

assertEquals(expected: 5, calculator.add(a: 2, b: 3), message: "Addition result mismatch");

assertNotEquals(unexpected: 6, calculator.subtract(a: 8, b: 3), message: "Subtraction result mismatch");

assertThrows(ArithmeticException.class, () -> calculator.divide(a: 5, b: 0), message: "Expected division by zero to throw");

assertDoesNotThrow(() -> calculator.divide(a: 10, b: 2), message: "No exception expected for valid division");

try {

calculator.divide(a: 10, b: 0);

fail("Division by zero should throw an exception");

} catch (ArithmeticException ignored) {}

assertTrue(condition: calculator.multiply(a: 2, b: 3) > 0, message: "Multiplication should return a positive result");

assertFalse(condition: calculator.add(a: 2, b: -5) > 0, message: "Addition result is not positive");

ACalculator sameCalculator = calculator;

ACalculator newCalculator = new ACalculator();

assertSame(calculator, sameCalculator, message: "Expected same object reference");

assertNotSame(calculator, newCalculator, message: "Expected different object references");

int[] expectedArray = {2, 4, 6};

int[] actualArray = {calculator.multiply(a: 1, b: 2), calculator.multiply(a: 2, b: 2), calculator.multiply(a: 3, b: 2)};

assertArrayEquals(expectedArray, actualArray, message: "Arrays do not match");

List<String> expectedLines = List.of("Line 1", "Line 2", "Line 3");

List<String> actualLines = List.of("Line 1", "Line 2", "Line 3");

assertLinesMatch(expectedLines, actualLines, message: "Lines do not match");

assertTimeout(Duration.ofMillis(100), () -> calculator.add(a: 1, b: 2), message: "Operation took too long");

}

assertThrows

- In JUnit testing, `assertThrows` is a method used to verify that a specific exception is thrown during the execution of a block of code.
- It helps ensure that code behaves as expected when it encounters exceptional situations, such as when a method throws an error or an exception is expected.
- **expectedType**: The class type of the exception you expect to be thrown. For example, `IllegalArgumentException.class`.
- **executable**: A lambda expression or method reference that represents the code which is expected to throw the exception.

Lambda Expressions

- Provide a clear and concise way to represent a method (or a function) that can be passed around and executed.
- Primarily used to define the implementation of a functional interface in a more compact and readable manner.
- The syntax of a lambda expression consists of three parts:
 - **Parameters:** The parameters (input) to the method.
 - **Arrow Token (->):** Separates the parameter list from the body.
 - **Body:** The code that will be executed by the lambda expression.
 - (parameters) -> expression

```
() -> System.out.println("Hello, World!");  
(x) -> x * 2;  
(x, y) -> x + y;
```

assertThrows(Cont.)

```
assertThrows(ArithmeticException.class, () -> calculator.divide( a: 5, b: 0), message: "Expected division by zero to throw");
assertThrows(ArithmeticException.class, new Executable() {
    @Override
    public void execute() throws Throwable {
        calculator.divide( a: 5, b: 0);
    }
}, message: "Expected division by zero to throw");
assertDoesNotThrow(() -> calculator.divide( a: 10, b: 2), message: "No exception expected for valid division");
```

```
assertThrows(ArithmeticException.class, () -> calculator.divide( a: 5, b: 0), message: "Expected division by zero to throw");
assertDoesNotThrow(() -> calculator.divide( a: 10, b: 2), message: "No exception expected for valid division");
```

assertThrows(Cont.)

- Lambda expressions in `assertThrows` allow for lazy evaluation and ensure that the code that might throw an exception is only executed at the right time.
- A normal function call would result in immediate evaluation, causing the exception to potentially be thrown before `assertThrows` can capture it, leading to incorrect test behavior.

assertTimeout

- Verify that a block of code finishes executing within a specified duration.
- Useful for testing the performance of your code or ensuring that certain operations don't take too long.
- If the block of code exceeds the allowed time, the test will fail.

```
assertTimeout(Duration.ofMillis(1), () -> calculator.add( a: 1, b: 2), message: "Operation took too long");
assertTimeout(Duration.ofMillis(1), () -> {
    //Thread.sleep(2);
    calculator.add( a: 1, b: 2);
});
```

Here, `Thread.sleep(milliseconds)` pauses the execution of the current thread for the specified number of milliseconds, allowing other threads to execute during that time.

assertLinesMatch

- Assert that two collections of lines match according to a specific condition.
- Useful for comparing collections of strings(e.g., List, Set, etc.).

```
List<String> expectedLines = List.of("Line 1", "Line 2", "Line 3");  
List<String> actualLines = List.of("Line 1", "Line 2", "Line 3");  
assertLinesMatch(expectedLines, actualLines, message: "Lines match");
```

Parameterized Tests

- Allow to run the same test with different sets of inputs.
- Useful for testing functions with multiple input-output combinations efficiently.
- JUnit provides various ways to supply parameters to tests, including:
 - **@ValueSource**
 - **@CsvSource**
 - **@CsvFileSource**
 - **@MethodSource**

Parameterized Tests(Cont.)

- **@ValueSource**
 - Used to provide a single array of values as input to a test.
 - It supports primitive types and strings.
- **@CsvSource**
 - Allow to supply multiple arguments (comma-separated values) for testing methods with multiple parameters.
- **@CsvFileSource**
 - Loads parameters from an external CSV file. This is useful for managing large datasets.
- **@MethodSource**
 - Supplies arguments from a static method within the same or a different class.
 - Supports complex parameter generation.

```
@ParameterizedTest
@ValueSource(ints = {1, 2, 3, 4, 5})
void testIsPositive(int number) {
    assertTrue( condition: number > 0);
}

@ParameterizedTest
@ValueSource(strings = {"Hello", "World"})
void testNotEmpty(String text) {
    assertNotNull(text);
    assertFalse(text.isEmpty());
}
```

@ValueSource

```

@ParameterizedTest
@CsvSource({
    "2, 3, 5",
    "3, 5, 8",
    "4, 6, 10"
})
void testAddition(int a, int b, int sum) { assertEquals(sum, actual: a + b); }

```

```

@ParameterizedTest
@CsvSource({"apple", "orange", "mango"})
void testIsNotEmpty(String fruit) {
    assertNotNull(fruit);
    assertFalse(fruit.isEmpty());
}

```

@CsvSource

```

@ParameterizedTest
@CsvSource({
    "hello, world, hello@example.com",
    "java, test, java@example.com",
    "unit, test, unit@example.com"
})
void testUserDetails(String firstName, String lastName, String email) {
    assertNotNull(firstName);
    assertNotNull(lastName);
    assertTrue(email.contains("@"));
}

```

```
@ParameterizedTest
@CsvFileSource(resources = "/data.csv", numLinesToSkip = 0)
void testAdditionFromFile(int a, int b, int sum) { assertEquals(sum, actual: a + b); }

@ParameterizedTest
@CsvFileSource(resources = "/data.csv", numLinesToSkip = 3)
void testUserDetailsFromFile(String firstName, String lastName, String email) {
    assertNotNull(firstName);
    assertNotNull(lastName);
    assertTrue(email.contains("@"));
}
```

@CsvFileSource

data.csv ×	
1	2, 3, 5
2	3, 5, 8
3	4, 6, 10
4	hello, world, hello@example.com
5	java, test, java@example.com
6	unit, test, unit@example.com
7	

@MethodSource

```
static Stream<Arguments> provideNumbers() {  
    return Stream.of(  
        Arguments.of(2, 3, 5),  
        Arguments.of(3, 5, 8),  
        Arguments.of(4, 6, 10)  
    );  
}
```

```
@ParameterizedTest  
@MethodSource("provideNumbers")  
void testSummation(int a, int b, int sum) { assertEquals(sum, actual: a + b); }
```

```
static Stream<Arguments> provideStrings() {  
    return Stream.of(  
        Arguments.of("Hello", "World", "HelloWorld"),  
        Arguments.of("Java", "Programming", "JavaProgramming"),  
        Arguments.of("Unit", "Test", "UnitTest")  
    );  
}
```

```
@ParameterizedTest  
@MethodSource("provideStrings")  
void testConcatenation(String str1, String str2, String result) { assertEquals(result, actual: str1 + str2); }
```

- In Java, a *Stream* is a sequence of elements that can be processed sequentially or in parallel.
- `Arguments` is a class in JUnit that is used to pass multiple test parameters to a parameterized test. It allows you to supply multiple input values in a structured way.
- `Arguments.of(value1, value2, ...)` is used to create a set of arguments for a single test case.
- JUnit executes the test method separately for each set of arguments.

Parameterized Tests(Cont.)

- Use `@ValueSource` for simple, single-parameter tests.
- Use `@CsvSource` for inline multi-parameter tests.
- Use `@CsvFileSource` for large datasets stored in external files.
- Use `@MethodSource` for dynamically generated or complex parameters.

Mockito

- Mockito is a popular Java-based framework used for unit testing by creating mock objects.
- It allows developers to isolate and test specific components of an application by simulating dependencies, ensuring that tests are independent and reliable.
- Dependency: mockito-core

Why Use Mockito?

- Isolates unit tests from external dependencies.
- Simplifies the testing of complex interactions.
- Improves test readability and maintainability.
- Reduces the need for real database connections or network calls in unit tests.

Mock and Stub

- **Mock:** A mock is a simulated object that mimics the behavior of a real object in controlled ways.
- **Stub:** A stub is a method or object that provides predefined responses to method calls made during testing.
- **Mocks** are used to verify interactions. **Stubs** are used to provide specific behavior or return values.

Annotations and Mockito Methods

- `@Mock`: Creates a mock object.
- `@InjectMocks`: Injects mock dependencies into a class.
- `@BeforeEach`: Initializes mocks before each test.
- `when().thenReturn()`: Defines behavior for a method call.
- `verify()`: Checks if a method was called with expected parameters.

Mockito

```
@ExtendWith(MockitoExtension.class)
class CalculatorServiceTest {
```

```
    @Mock
```

```
    private Calculator calculator;
```

```
    @InjectMocks
```

```
    private CalculatorService calculatorService;
```

```
    @BeforeEach
    void setUp() {
        MockitoAnnotations.openMocks(this);
    }
```

```
    @Test
```

```
    void performAddition() {
        when(calculator.add( a: 5, b: 3)).thenReturn(8); //stub the add method to return 8
        //assertEquals(5, calculatorService.performAddition(2, 3));
        assertEquals( expected: 8, calculatorService.performAddition( a: 5, b: 3));

        verify(calculator).add( a: 5, b: 3);
        verify(calculator, times( wantedNumberOfInvocations: 1)).add( a: 5, b: 3);
        verify(calculator, never()).add( a: 3, b: 5);
    }
```

```
public class CalculatorService {
    private Calculator calculator;

    public CalculatorService(Calculator calculator) {
        this.calculator = calculator;
    }

    public int performAddition(int a, int b) {
        return calculator.add(a, b);
    }
}
```